

RAG Application — Sample Knowledge Document

This sample PDF is designed for testing a Retrieval-Augmented Generation (RAG) application. It contains structured information that can be loaded, split into chunks, embedded, stored in a vector database such as Qdrant, and retrieved when answering user questions.

1. What is RAG?

Retrieval-Augmented Generation, commonly called RAG, is an architecture that combines information retrieval with a large language model. Instead of relying only on the model's training knowledge, the application first searches a knowledge base for relevant information. The retrieved content is then provided to the language model as context for generating an answer.

2. Typical RAG Pipeline

Step	Description
1. Load	Read source documents such as PDFs, text files, web pages, or documentation.
2. Split	Break documents into smaller chunks so relevant passages can be retrieved efficiently.
3. Embed	Convert each chunk into a numerical vector representation using an embedding model.
4. Store	Store vectors and their metadata in a vector database such as Qdrant.
5. Retrieve	Convert a user's question into a vector and search for similar document chunks.
6. Generate	Send the question and retrieved context to an LLM to produce the final answer.

3. Example RAG Project

Consider a FastAPI application called RAG Assistant. The application allows users to upload company documents and ask questions about them. Documents are processed into chunks, embedded using an embedding model, and stored in Qdrant. When a user asks a question, the application retrieves the most relevant chunks and supplies them to an LLM.

4. Example Knowledge

Product: RAG Assistant

Version: 1.0

Maximum document size: 20 MB

Supported formats: PDF, TXT, DOCX

Vector database: Qdrant

API framework: FastAPI

Embedding provider: OpenAI-compatible embedding model

Default chunk size: 500 characters

Default chunk overlap: 50 characters

5. Important RAG Concepts

Chunking: Splitting a large document into smaller pieces. Good chunking helps retrieval return focused and useful context.

Embeddings: Numerical representations of text that allow semantically similar pieces of text to be found.

Vector Database: A database optimized for storing vectors and searching for similar vectors.

Similarity Search: A search method that finds document chunks whose embeddings are most similar to the query embedding.

Metadata: Additional information stored with a chunk, such as filename, page number, document ID, or source.

Context: Retrieved document content supplied to the language model so it can answer using the application's knowledge base.

6. Sample Questions for Testing Retrieval

- What vector database is used by the RAG Assistant?
- What document formats are supported?
- What is the default chunk size?
- What is the default chunk overlap?
- What framework is used to build the API?
- What are the main stages of a RAG pipeline?
- Why are documents split into chunks?
- What is the purpose of embeddings?

7. Expected RAG Behavior

When a user asks a question covered by this document, the retriever should return the relevant chunk containing the answer. The language model should use that retrieved context rather than inventing information. For questions that are not supported by the retrieved knowledge, the application should clearly indicate that the answer was not found in the available documents.

End of sample document — suitable for experimenting with PDF loading, text splitting, embeddings, Qdrant storage, similarity search, and FastAPI-based RAG endpoints.